

R4 CMS – Guida ai Plugin

Questa guida descrive **come sono gestiti i plugin** nel CMS, quali file sono coinvolti, come confezionare uno **ZIP installabile**, come vengono caricati gli **asset** (JS/CSS) sia in **frontend** sia in **admin**, e come creare un **blocco Page Builder** di tipo `plugin:*`.

1) Panoramica di architettura

Componenti principali

- **Model:** `App\Models\Plugin` (cast `manifest` → array, flag `enabled`).
- **Service:** `App\Services\PluginManager`
- Installa da ZIP, sposta i file nella cartella plugin, copia gli asset pubblici, costruisce liste asset **admin** e **frontend**, espone registro blocchi.
- **Controller Admin:** `App\Http\Controllers\Admin\PluginController`
- Upload/Install, Enable/Disable, Destroy (rimozione file + record DB).
- **Config attesa** (valori letti da `config('plugins.*')`):
- `path` (cartella codice plugin), `public_path` (destinazione asset pubblici),
- `manifest` (es. `plugin.json`), `public_dir` (es. `public`),
- `admin_entry` (default `admin.js`), `view_entry` (default `view.js`),
- `web_path` (prefisso URL pubblico, es. `/plugins`).

Flusso di esecuzione

1. **Installazione** (Admin → upload ZIP): estrazione temporanea, lettura **manifest**, validazione `slug`, copia in `plugins.path/{slug}`, copia asset `{plugin}/public/` → `/public/plugins/{slug}/`.
2. **Attivazione:** l'admin abilita (`enabled=true`). Solo i plugin **abilitati**:
3. vengono inclusi negli asset **Admin** e **Frontend**;
4. espongono i **blocchi** per il Page Builder (via manifest `blocks`).
5. **Rendering:**
6. **Admin Page Builder:** mostra i bottoni dei blocchi `plugin:*` e opzionalmente esegue JS `admin.js` del plugin.
7. **Frontend:** il renderer stampa un placeholder `<div class="pb-plugin" data-type="plugin:..." data-payload='...'>`; un ciclo di **mount** cerca in `window.BuilderPlugins[type]` una funzione `mount(el, data)` o `renderView(data)` e monta il plugin.

2) Struttura file (attesa) di un plugin

```
my-plugin/
├─ plugin.json          # manifest (obbligatorio)
├─ public/              # asset pubblici copiati in /public/plugins/
│   └─ {slug}
│       └─ view.js      # entry frontend
```

```

|   └─ view.css                # stile frontend (opzionale, consigliato)
|   └─ ... (icone, immagini, altri js/css)
└─ admin.js                   # entry admin (opzionale ma consigliato)

```

Nota: il file `admin.js` può anche stare nella `public/` se preferisci, l'importante è che il **percorso** usato in `plugin.json` sia coerente e che il file esista nel FS pubblico finale (vedi §5).

3) Manifest – schema e esempio

Percorso/Nome: `plugin.json` (case-insensitive, cercato ricorsivamente nello ZIP).

Schema minimo

```

{
  "slug": "r4-logos-carousel",           // [a-z0-9-] obbligatorio
  "name": "R4 Carosello Loghi",
  "version": "1.1.0",
  "author": "R4Software",
  "blocks": [
    { "type": "plugin:r4-logos-carousel", "label": "Carosello Loghi",
  "icon": "bi-sliders", "add_columns": 12 }
  ],
  "assets": {
    "front": ["view.js", "view.css"] // relativi a {plugin}/public/
  },
  "admin_entry": "admin.js"           // relativo a {plugin}/public/ oppure
  root, vedi §5
}

```

Campi supportati

- `slug`: **string** requerido, regex `^[a-z0-9-]+$`.
- `blocks[]`: definisce i blocchi che l'editor può aggiungere:
- `type`: deve iniziare con `plugin:` (es. `plugin:r4-logos-carousel`).
- `label`, `icon` (Bootstrap Icons), `add_columns` (colonne iniziali nel builder).
- `assets.front[]`: elenco di file CSS/JS da includere in **frontend** (e, per anteprima, anche in **admin**, v. §5.2).
- `admin_entry`: file JS da includere nell'**area admin** (es. per `renderEditor`).
- `version`, `author`: liberi.

4) File coinvolti lato app e responsabilità

- `App\Http\Controllers\Admin\PluginController`
- `upload()`: valida ZIP e chiama `PluginManager::installFromZip()`.
- `enable()/disable()`: toglia `enabled` nel DB.

- `destroy()` : elimina cartelle **codice** e **pubbliche** e rimuove record.
- `App\Services\PluginManager`
- Lettura config percorsi (`path`, `public_path`, ...).
- `installFromZip()`
 - Estrae, individua `plugin.json`, valida `slug`, sposta nella dir definitiva, **copia asset pubblici**.
 - Upsert su `Plugin` con `manifest`.
- `buildBlocksRegistry()`
 - Raccoglie i blocchi dai manifest dei plugin **abilitati**.
- `adminAssets()` → [`'css'=>[]`, `'js'=>[]`]
 - Inietta prima `admin_entry` se esiste, poi eventuali asset `assets.front` (CSS/JS) per avere anteprime coerenti in builder.
- `frontAssets()` → `string[]`
 - Elenco piatto (CSS/JS) da **manifest.assets.front** per i plugin **abilitati**.
- `frontendAssets()` → `string[]`
 - Fallback più rigido: include solo `view_entry` (es. `view.js`) se presente sul FS pubblico. Usato se `assets.front` non è definito.
- `assetUrlVersioned()`
 - Genera URL con query `?v=mtime` quando il file è locale (cache-busting).

• Blade pubblici

- `resources/views/layouts/app.blade.php` (Template Pubblico)
 - Alla fine del `<body>`

```
$pluginFrontAssets = $pm->frontAssets();
if (empty($pluginFrontAssets)) { $pluginFrontAssets = $pm->frontendAssets(); }
// stampa tag <link>/<script>
```

- **Consiglio:** centralizzare **qui** l'inclusione asset plugin, ed evitare di ripeterla altrove (vedi §9 punto A).
- `resources/views/pages/show.blade.php`
 - Oggi ricalcola e `@push` CSS/JS. **Suggerito:** rimuovere questa sezione per evitare doppi inserti quando già si includono dal layout pubblico.

• Renderer Page Builder (frontend)

- `resources/views/partials/page_renderer.blade.php`
 - Emette gli elementi `<div class="pb-plugin" data-type="plugin:..." data-payload='...'>`.
 - JS di supporto cerca `window.BuilderPlugins[type]` e chiama `mount(el, data)` oppure usa `renderView(data)`.

- Effettua un **mount-loop** per attendere il caricamento JS dei plugin (fino a ~6s).

- **Editor Page Builder (admin)**

- `resources/views/admin/pages/edit.blade.php`
 - Riceve server-side `window.__PLUGIN_BLOCKS__ = ...` con la lista costruita da `PluginManager::buildBlocksRegistry()`.
 - Mostra bottoni "Plugin ..." nella **palette** del blocco.
 - Se il JS del plugin espone `renderEditor(sec, block)`, lo usa per mostrare un pannello ad-hoc nel modal impostazioni.

5) Inclusione degli asset (CSS/JS)

5.1 Frontend

- **Preferito:** dichiarare tutti gli asset frontend nel manifest → `assets.front`.
- Il template pubblico chiama `$pm->frontAssets()` e inietta **in coda** alla pagina i CSS/JS con `defer` (no `type="module"`).
- Fallback: se `assets.front` è vuoto, `$pm->frontendAssets()` cerca `view.js` esistente per ciascun plugin abilitato.

5.2 Admin

- `$pm->adminAssets()` ritorna **sempre** `['css'=>[], 'js'=>[]]` e **inserisce prima** `admin_entry`, poi eventuali file da `assets.front` (utile per avere lo stesso CSS del plugin in anteprima).
- **Snippet da inserire nel layout admin** (se non già presente):

```
@php($pm = app(\App\Services\PluginManager::class))
@php($assets = $pm->adminAssets())
@foreach($assets['css'] as $href)
    <link rel="stylesheet" href="{{ $href }}">
@endforeach
@foreach($assets['js'] as $src)
    <script src="{{ $src }}" defer></script>
@endforeach
```

Suggerimento: in admin, se qualche plugin ha JS "invasivo", si può decidere di includere **solo i CSS** dai `front` lasciando i JS al solo frontend. Questo dipende dal plugin.

6) API attesa dal JS del plugin

Il renderer e l'editor cercano oggetti in **global**:

```
// chiave = manifest.blocks[i].type
window.BuilderPlugins = window.BuilderPlugins || {};
window.BuilderPlugins["plugin:r4-qualcosa"] = {
```

```

// FRONTEND (pagina pubblica)
mount(el, data) {
  // Inietti DOM e fai il bind di eventuali listener.
  // el è il <div.pb-plugin ...>
  // data è l'oggetto serializzato in page content (b.data)
},
// opzionale (frontend): in alternativa a mount
renderView(data) {
  // ritorna stringa HTML.
  return '<div>...</div>';
},
// ADMIN (editor)
renderEditor(sec, block) {
  // ritorna stringa HTML del pannello impostazioni "Contenuto"
  // (il modal dell'editor la inserisce direttamente)
  return '<div class="alert alert-info">Impostazioni custom...</div>';
}
};

// (facoltativo) - emetti un evento quando il tuo JS è carico
document.dispatchEvent(new Event('plugins:ready'));

```

Raccomandazioni

- Evitare moduli ES: **non** usare `type="module"` (l'app usa `<script defer>` classico).
- Evitare dipendenze globali non necessarie; namespacciare sotto `window.BuilderPlugins`.
- Non affidarsi a jQuery (se non già presente nel progetto).

7) Esempio completo – “R4 Carosello Loghi”

plugin.json

```

{
  "slug": "r4-logos-carousel",
  "name": "R4 Carosello Loghi",
  "version": "1.1.0",
  "author": "R4Software",
  "blocks": [
    { "type": "plugin:r4-logos-carousel", "label": "Carosello Loghi",
  "icon": "bi-sliders", "add_columns": 12 }
  ],
  "assets": { "front": ["view.js", "view.css"] },
  "admin_entry": "admin.js"
}

```

public/view.css (snippet base per disporre i loghi in orizzontale)

```
.r4lc { position: relative; overflow: hidden; }
.r4lc-track {
  display: flex; align-items: center; gap: var(--r4lc-gap, 24px);
  will-change: transform; animation: scroll-x var(--r4lc-speed, 30s) linear
infinite;
}
.r4lc img { height: var(--r4lc-height, 72px); width: auto; display: block;
object-fit: contain; }
@keyframes scroll-x { from { transform: translateX(0); } to { transform:
translateX(-50%); } }
```

public/view.js (frontend)

```
(function(){
  const TYPE = 'plugin:r4-logos-carousel';
  window.BuilderPlugins = window.BuilderPlugins || {};
  window.BuilderPlugins[TYPE] = {
    mount(el, data){
      try{
        const cfg = data || {}; // es: { items:[{src,alt}], height, gap,
speed
        const h = parseInt(cfg.height || 72, 10);
        const gap = parseInt(cfg.gap || 24, 10);
        const speed = parseInt(cfg.speed || 30, 10);
        const items = Array.isArray(cfg.items) ? cfg.items : [];
        el.innerHTML = `
          <div class="r4lc" style="--r4lc-height:${h}px; --r4lc-gap:${gap}
px; --r4lc-speed:${speed}s">
            <div class="r4lc-track">${items.concat(items).map(i => ``).join('')}</div>
            </div>`;
        }catch(err){
          el.innerHTML = `<div class="alert alert-warning mb-0">Errore: $
{String(err)}</div>`;
        }
      }
    };
    document.dispatchEvent(new Event('plugins:ready'));
  })();
```

admin.js (area admin – pannello impostazioni del blocco)

```
(function(){
  const TYPE = 'plugin:r4-logos-carousel';
  window.BuilderPlugins = window.BuilderPlugins || {};
  const API = window.BuilderPlugins[TYPE] || (window.BuilderPlugins[TYPE] =
{});
```

```

API.renderEditor = function(sec, block){
  const d = block.data = block.data || { items:[], height:72, gap:24,
speed:30 };
  const list = (d.items||[]).map((it, i) => `
    <div class="row g-2 align-items-center mb-1">
      <div class="col"><input class="form-control form-control-sm" value="$
{it.src||''}" oninput="updatePluginField('${sec.id}','$
{block.id}','data.items.${i}.src', this.value)"></div>
      <div class="col-auto"><button type="button" class="btn btn-sm btn-
outline-danger" onclick="(function(){ const b=findBlockById('${
{block.id}').blk; b.data.items.splice(${i},1); renderBuilder(); })()"><i
class="bi bi-trash"></i></button></div>
    </div>`).join('');

  return `
    <div class="mb-2">
      <label class="small">Altezza loghi (px)</label>
      <input type="number" class="form-control form-control-sm" value="$
{d.height||72}" oninput="updatePluginField('${sec.id}','$
{block.id}','data.height', parseInt(this.value||72,10));">
    </div>
    <div class="mb-2">
      <label class="small">Spaziatura (px)</label>
      <input type="number" class="form-control form-control-sm" value="$
{d.gap||24}" oninput="updatePluginField('${sec.id}','$
{block.id}','data.gap', parseInt(this.value||24,10));">
    </div>
    <div class="mb-2">
      <label class="small">Velocità (s)</label>
      <input type="number" class="form-control form-control-sm" value="$
{d.speed||30}" oninput="updatePluginField('${sec.id}','$
{block.id}','data.speed', parseInt(this.value||30,10));">
    </div>
    <div class="mb-2">
      <div class="d-flex align-items-center justify-content-between mb-1">
        <label class="small mb-0">Loghi</label>
        <button type="button" class="btn btn-sm btn-soft"
onclick="(function(){ const b=findBlockById('${block.id}').blk;
(b.data.items=b.data.items||[]).push({src:'',alt:''}); renderBuilder(); })
()"><i class="bi bi-plus-lg"></i> Aggiungi</button>
      </div>
      ${list || '<div class="text-muted small">Nessun logo</div>'}
    </div>`;
  };

  document.dispatchEvent(new Event('plugins:ready'));
})();

```

8) Come confezionare lo ZIP installabile

1. Prepara la struttura del plugin **come da §2**.
 2. Verifica che `plugin.json` esista e abbia `slug` valido.
 3. Metti tutti gli asset dentro `public/` (verranno copiati in `/public/plugins/{slug}/`).
 4. Comprimi **il contenuto della cartella** (non la cartella parent) in `mio-plugin.zip`.
 5. In **Admin → Plugin** carica lo ZIP, poi **Abilita**.
-

9) Audit & miglioramenti consigliati

A) Doppio inserimento asset in public

- Oggi sia `layouts/app.blade.php` sia `pages/show.blade.php` iniettano asset dei plugin.
Scegli una sola via. Consigliato **solo dal layout pubblico** per uniformità.

B) Admin: valutare front JS

- `adminAssets()` include anche i JS presenti in `assets.front`. Se qualche plugin ha side effect nell'editor, si può limitarne l'inclusione in admin ai **solì CSS** (piccola modifica nel service).

C) Evento “plugins:ready”

- Già gestito nel builder/renderer; raccomandato emetterlo alla fine dei file `view.js` / `admin.js` del plugin.

D) Validazione ZIP

- `installFromZip()` è robusto; per ambienti multi-tenant si può aggiungere una **white-list** di estensioni ammesse in `public/` (escludere `.php`).

E) Sicurezza

- I plugin eseguono JS nel frontend. Consentire upload/abilitazione solo ad admin fidati. Valutare una firma o provenienza interna.
-

10) Linea guida d'uso (per editor di contenuti)

1. Vai su **Pagine → Modifica**.
2. Clicca **Aggiungi Blocco** nella sezione desiderata.
3. Nella palette, scegli il tuo **Plugin** (icona puzzle / nome definito nel manifest).
4. Clicca l'icona **impostazioni** del blocco e compila i campi del pannello (forniti dal plugin via `renderEditor`).
5. **Salva** o **Pubblica** la pagina.

Se il plugin non si monta in anteprima, aspetta qualche istante: il sistema riprova finché gli asset non sono caricati.

11) Linea guida dev rapida (checklist)

- [] slug valido (`^[a-z0-9-]+$`).
 - [] plugin.json con blocks[], assets.front[], admin_entry se serve.
 - [] Tutto il necessario in public/ (JS/CSS/icone).
 - [] view.js registra window.BuilderPlugins[TYPE] e implementa mount() o renderView() .
 - [] admin.js (se usato) implementa renderEditor() .
 - [] Nessun type="module" nei tag <script> .
 - [] Test: installa, abilita, aggiungi il blocco, verifica che il mount avvenga e che i dati serializzati in data funzionino.
-

12) FAQ / Troubleshooting

- **"Plugin ... in caricamento..." non scompare**
 - Controlla in Console se window.BuilderPlugins["tipo"] esiste.
 - Verifica percorsi in assets.front / admin_entry e che i file siano finiti in /public/plugins/{slug}/ .
 - Assicurati che gli asset siano inclusi **una sola volta** (vedi §9.A).
 - **Loghi in verticale nel carosello**
 - Assicurati che il contenitore usi display:flex (vedi CSS in §7) e che le immagini abbiano height:auto/contain coerente.
 - **Asset non "versionati"**
 - assetUrlVersioned() aggiunge ?v=mtime se il file esiste nel FS pubblico. Se non vedi la query, controlla la posizione reale del file.
 - **Blocco non visibile in palette**
 - Il plugin deve essere **abilitato** e il manifest.blocks[] deve avere type che inizia con plugin: .
-

Questa guida è allineata al codice fornito e ai comportamenti del PluginManager, dei Blade pubblici e dell'editor.